SYSTEM DESIGN: THROUGHPUT

Easy English lecture notes | Based on the supplied Hindi video transcript

1. What is throughput?

Throughput is the amount of work completed, or information transferred, by a system per unit of time. It answers: *How much work can the system handle in a second?*

Lecture example: If a creator produces 100 videos in 100 days, their average throughput is **1 video/day**.

Common system measures include **requests/second (RPS)**, **transactions/second (TPS)**, and **bits/second (bps)** for data transfer. The unit depends on what is being measured.

```
Throughput = completed work / elapsed time
```

2. Monolithic vs distributed architecture

The lecture explains that distributed systems can split work across multiple machines and can scale horizontally, which may allow higher throughput than a single constrained deployment.

Aspect	Monolithic system	Distributed system
Deployment	Components typically deployed together	Components may run on separate machines
Scaling	Can scale vertically or replicate whole app	Can scale components or services independently
Throughput	Depends on available resources and bottlenecks	Can improve with parallelism and scaling
Trade-off	Fewer internal network hops	Extra network calls and coordination

Important nuance: Distributed architecture does not automatically deliver higher throughput. Performance depends on design, resource limits, database capacity, network overhead, and bottlenecks.

3. Horizontal scaling and load balancing

- **Horizontal scaling:** Add more servers or instances to share work.
- **Load balancer:** Distribute incoming requests among available instances.
- **Round robin:** A simple strategy that sends successive requests to different servers in turn.

```
Client requests -> Load balancer -> Server A / Server B / Server C
```

4. Why throughput can be low

- **High latency:** Slow requests can reduce the rate of completed work, particularly when concurrency is limited.
- **Protocol overhead:** Handshakes, extra communication, and repeated round trips consume time and resources.

- **Congestion:** Too many requests arriving together can saturate the network, CPU, database, or request queues.

5. Ways to improve throughput

- **Caching:** Reuse previously computed or fetched results, reducing repeated backend work.
- **CDN:** Serve cacheable static content from geographically distributed edge servers, reducing origin load.
- **Distributed deployment:** Divide suitable workloads among multiple servers or services.
- **Load balancing:** Spread requests across healthy instances.
- **Upgrade resources:** Increase CPU, memory, bandwidth, or database capacity when those are bottlenecks.

6. Throughput vs latency

Latency	How long one request takes to complete	Example: 150 ms/request
Throughput	How many requests complete per unit time	Example: 800 requests/second

They are related but not interchangeable. A system can serve many requests concurrently even if individual requests have noticeable latency.

7. Java / Spring Boot example

Imagine a Spring Boot API that processes 600 requests in 10 seconds:

```
Throughput = 600 / 10 = 60 requests/second
```

If caching reduces repeated database queries, the API may process more requests in the same period. Adding instances behind a load balancer can also increase capacity, if the database and other shared resources can keep up.

8. Quick interview revision

- **Define throughput:** Work completed per unit of time.
- **How is it measured?** RPS, TPS, bps, or other task-specific units.
- **Why can distributed systems help?** They enable workload distribution and horizontal scaling.
- **Name three throughput bottlenecks:** Latency, protocol overhead, congestion.
- **Name five improvements:** Caching, CDN, scaling/distribution, load balancing, resource upgrades.

Source note: These notes paraphrase the supplied lecture, with brief clarifications where the transcript oversimplifies architecture and performance.